Natural Language Processing

UNIVERSITÄT BONN

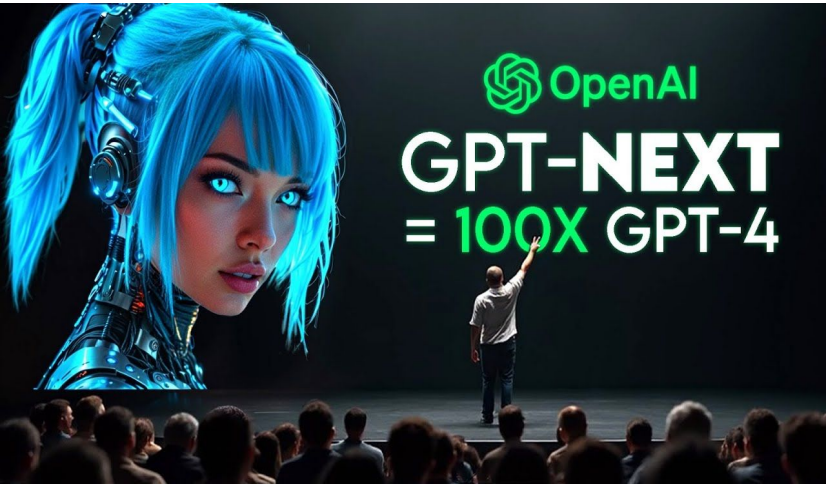


Lecture 2: Word Embeddings

- Espresso? But I ordered a cappuccino!
- Don't worry, the cosine distance between them is so small that they are almost the same thing.

Content Overview





- 30.10. Intro, types of NLP tasks, preprocessing
- 6.11. Word vectors, embeddings
- 13.11. Neural nets - RNNs, attention, ELMo
- 20.11. Transformer overall, BERT
- **27.11.** BERT continued (specifics), BPE
- 4.12. Bertology, GPT2, T5
- **11.12.** LLM Evaluation
- 18.12. Decoding strategies
- 8.1. from "Transformers" to "LLMs", instruction tuning, reasoning, alignment
- 15.1. Multilingual/-modal LLMs, agents
- 22.1. Ongoing research topics
- 29.1. Last lecture - repetition for the exam

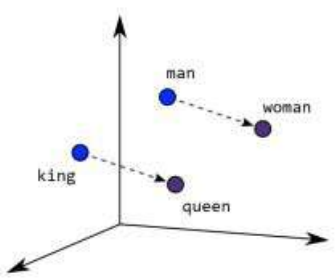
Words are related! (But how?)



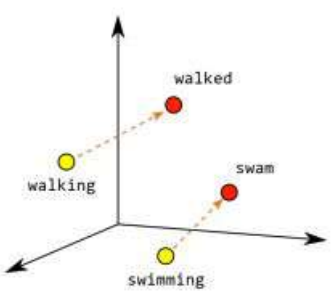
Today's goals



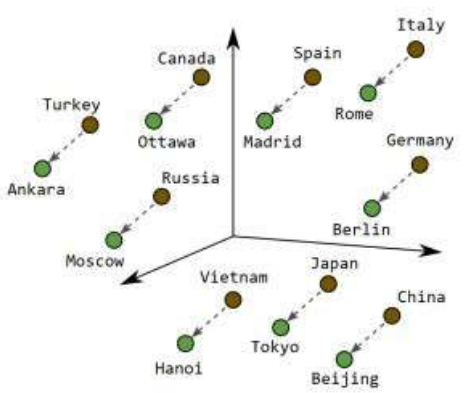
- Understand what word embeddings are
- Learn the main methods for creating them
- Understand configuration choices in methods
- Dealing with limitations:
  - Unknown words
  - multiple senses



Male-Female



Verb Tense



Country-Capital

Representing words

Text classification

$$P(y = 1 \mid x) = \sigma(\theta^{\mathsf{T}} w + b)$$

I like this movie. 👍

$w^{(1)}$ [0, 1, 0, 0, 0, ..., 1, ..., 1]

I don't like this movie. 👎

$w^{(2)}$ [0, 1, 0, 1, 0, ..., 1, ..., 1]

don't

Representing words in pre-neural NLP times (2010s)



In traditional NLP, we regard words as discrete symbols:

hotel, conference, motel — a localist representation

one 1, the rest 0's

Words can be represented by one-hot vectors:

hotel = [0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

motel = [0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0]

Vector dimension = number of words in vocabulary (e.g., 500,000)

Challenge: How to compute similarity of two words?

Even back then, we knew that words have context



Distributional hypothesis: words that occur in similar contexts tend to have similar meanings



J.R.Firth 1957

- "You shall know a word by the company it keeps"
- One of the most successful ideas of modern statistical NLP!

…government debt problems turning into	***banking***	*crises as happened in 2009…*
…saying that Europe needs unified	***banking***	*regulation to replace the hodgepodge…*
…India has just given its	***banking***	*system a shot in the arm…*

These context words will represent *banking*.

Even back then, we knew that words have context

Contexts for '*tea*':

establishments, besides two livery stables, a	tea They never boasted of Robert Acton, nor indulg
en things, because their methods of family to	tea at once. pose, which you carry so well
, as, indeed, A waiter comes in with the	tea. He places the tray on the table. Jasper
let me always remain here.' "I prefer weak	tea!" cried Daisy, and she went off with the
hell. I should think you had drunk enough	tea in Chin a. life; it is a failure,
. Not a bit. Come in and have some	tea. Stay to dinner. every year. Don't persist
responsibility. And greatly as we enjoyed our	tea Crusoe island. Then there's the religious diff
your naturally liking me. (She is and had	tea in the evening. Afraid though as he was
] Tell them I shan't be home to	tea, will you, LADY BRITOMART. I must get the
, that was Mr. McComas will not come to	tea, ma'am: he has gone to call upon
, or asked you to have a cup of	tea. It's not human. ugly woman must have
woman can hardly know one places Gilbey's	tea on the table before him]. The lady that
of my — my hopes.' BROADBENT. He'll want	tea. Let us have some. BURGE-LUBIN [_resolutely ge
: THE MANAGER. Can I take any order? Some	tea? would THE SHE-ANCIENT. Speak, Arjillax: you w
to Tramp.} Will you drink a sup of	tea with myself and the the happiest person in
are trying to sleep." the evening after your	tea. "Better still—then there you are!" And Streth
GUINNESS. I'll go get you some fresh	tea, ducky. [She takes up the its burden, is
sional men, artists, and even with laborers	tea services out and made the people who had
the Lutches and Mrs. Rance the attendance at	tea just in the right place on the west
she came over to the great house to	tea. She had let the proposal that she should
. The Baroness found it amusing to go to	tea; she dressed as if for dinner. The tea-
tea; she dressed as if for dinner. The	tea-table offered an anomalous and picturesque rep
would be dead in two years, as the	tea-table. Be serious, Felix. You forget that I

Contexts for '*coffee*':

prompt his an incident in my life as	coffee for breakfast. Of course, hes too _Two fig
ek her out all courteously, PETKOFF (over his	coffee and cigaret). I don't believe in going
could be done, too,' he remarked, sipping his	coffee. 'Bury him in some sort,' I explained. 'One
, of us. I should like a cup of	coffee. MICHAEL. If you'd come in better hours,
UKA (innocently). Perhaps you would like some	coffee, sir? DISCOVERY ANTICIPATED BY DIVINATION s
her in public because he has fallen head	coffee-colored heathens and pestilential white agi
manners for he was novels, broken backed,	coffee stained, torn and '''This was the last time
stretches her hand across the table for the	coffee pot.) welcome, an expression which drops in
ittle sitting-room, and cigarettes, after the	coffee, had been permitted by the ladies, and in
had just given me a pannikin of hot	coffee....Slapped it down there, on my chest—bange
a heavy roll coming; tried to save my	coffee, burnt my fingers....and fell out of my
I'll have a claret cup instead of	coffee. Put some first night that we've come
t of trouble travelling. And then, with fresh	coffee, a clean cup, and a brandy bottle on
. Your word had such weight with me!" fresh	coffee? He gave his friend a glance as to
wont press you. 'Try a weed with your	coffee. Local tobacco. The black coffee you get at

Distributional hypothesis

"tejuino"



C1: A bottle of ____ is on the table.

C2: Everybody likes ____.

C3: Don't have ____ before you drive.

C4: We make ____ out of corn.

Distributional hypothesis

C1: A bottle of ____ is on the table.

C2: Everybody likes ____.

C3: Don't have ____ before you drive.

C4: We make ____ out of corn.

	C1	C2	C3	C4
tejuino	1	1	1	1
loud	0	0	0	0
motor-oil	1	0	0	0
tortillas	0	1	0	1
choices	0	1	0	0
wine	1	1	1	0

"words that occur in similar contexts tend to have similar meanings"

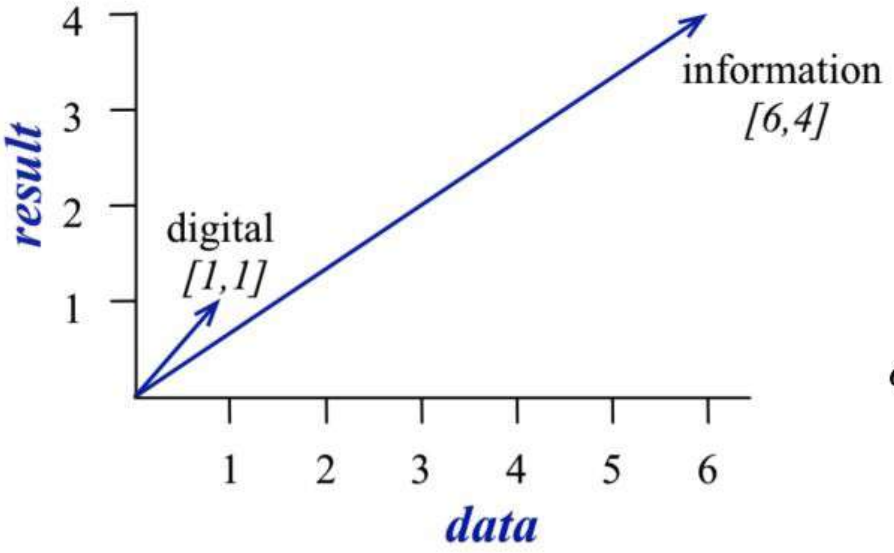
Word as vectors

- We'll build a new model of meaning focusing on similarity
  - Each word is a vector
  - Similar words are "nearby in space"

- A first solution: we can just use context vectors to represent the meaning of words!
  - word-word co-occurrence matrix:

	aardvark	computer	data	pinch	result	sugar	...
apricot	0	0	0	1	0	1	
pineapple	0	0	0	1	0	1	
digital	0	2	1	0	1	0	
information	0	1	6	0	4	0	

Words as vectors



$$cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$$

$$cos(\mathbf{u}, \mathbf{v}) = \frac{\sum_{i=1}^{V} u_i v_i}{\sqrt{\sum_{i=1}^{V} u_i^2}\sqrt{\sum_{i=1}^{V} v_i^2}}$$

What is the range of cos(·)?

Words as vectors

Problem: not all counts are equal, words can randomly co-occur

- Solution: re-weight by how likely it is for the two words to co-occur by simple chance
- PPMI = Positive Pointwise Mutual Information

$$\text{PPMI}(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

	computer	data	result	pie	sugar
cherry	2	8	9	442	25
strawberry	0	0	1	60	19
digital	1670	1683	85	5	4
information	3325	3982	378	5	13

	computer	data	result	pie	sugar
cherry	0	0	0	4.38	3.30
strawberry	0	0	0	4.10	5.51
digital	0.18	0.01	0	0	0
information	0.02	0.09	0.28	0	0

Words as vectors

Still, the vectors we get from
word-word occurrence matrix are sparse
(most are 0's) & long (vocabulary size)

- Alternative:

we want to represent words as
short (50-300 dimensional)
& dense (real-valued) vectors

$$\text{employees} = \begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 10.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$

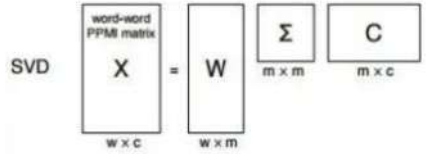


WHY?

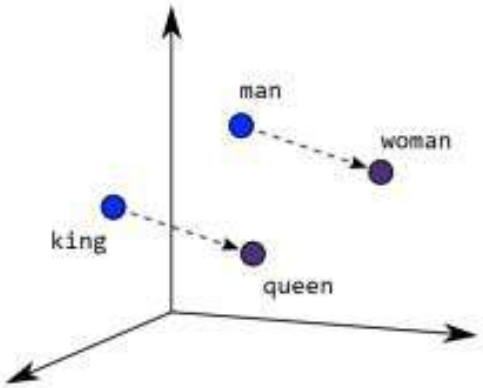
- Short vectors are easier to use in ML systems
- Dense vectors may generalize better
- They do better at capturing synonymy

Different methods for getting dense vectors

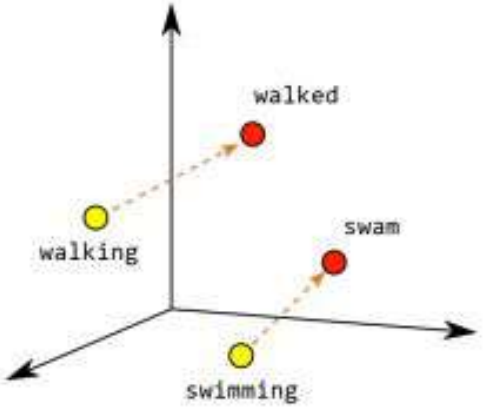
- Singular value decomposition (SVD)
- word2vec and friends: "learn" the vectors



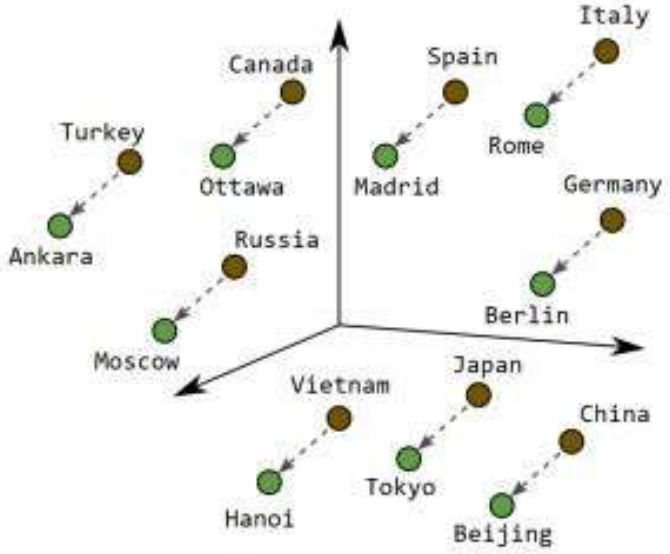
Word2vec



Male-Female



Verb Tense



Country-Capital

(Mikolov et al, 2013): Distributed Representations of Words and Phrases and their Compositionality



Word2vec + cosine similarity

word = “sweden”

Word	Cosine distance
norway	0.760124
denmark	0.715460
finland	0.620022
switzerland	0.588132
belgium	0.585835
netherlands	0.574631
iceland	0.562368
estonia	0.547621
slovenia	0.531408

Word2vec

- Input: a large text corpora, V, d
  - V: a pre-defined vocabulary
  - d: dimension of word vectors (e.g. 300)
  - Text corpora:
    - Wikipedia + Gigaword 5: 6B
    - Twitter: 27B
    - Common Crawl: 840B
- Output: $f : V \to \mathbb{R}^d$

$$v_{\text{cat}} = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ 0.276 \end{pmatrix} \quad v_{\text{dog}} = \begin{pmatrix} -0.124 \\ 0.430 \\ -0.200 \\ 0.329 \end{pmatrix}$$

$$v_{\text{the}} = \begin{pmatrix} 0.234 \\ 0.266 \\ 0.239 \\ -0.199 \end{pmatrix} \quad v_{\text{language}} = \begin{pmatrix} 0.290 \\ -0.441 \\ 0.762 \\ 0.982 \end{pmatrix}$$

(Mikolov et al, 2013): Distributed Representations of Words and Phrases and their Compositionality

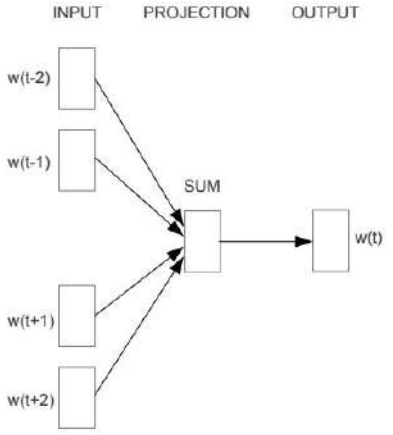


Two ways to think about Word2Vec:

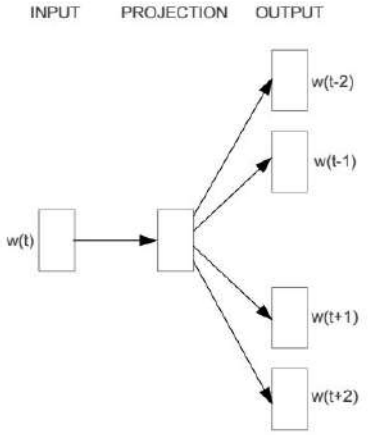
- a simplification of neural language models
- a binary logistic regression classifier

Variants of Word2Vec :

- Two different context representation options:
  - CBOW:
    Predict center word from (bag of) context words
  - Skip-Gram:
    Predict context (“outside”) words
    (position independent) given center word



CBOW



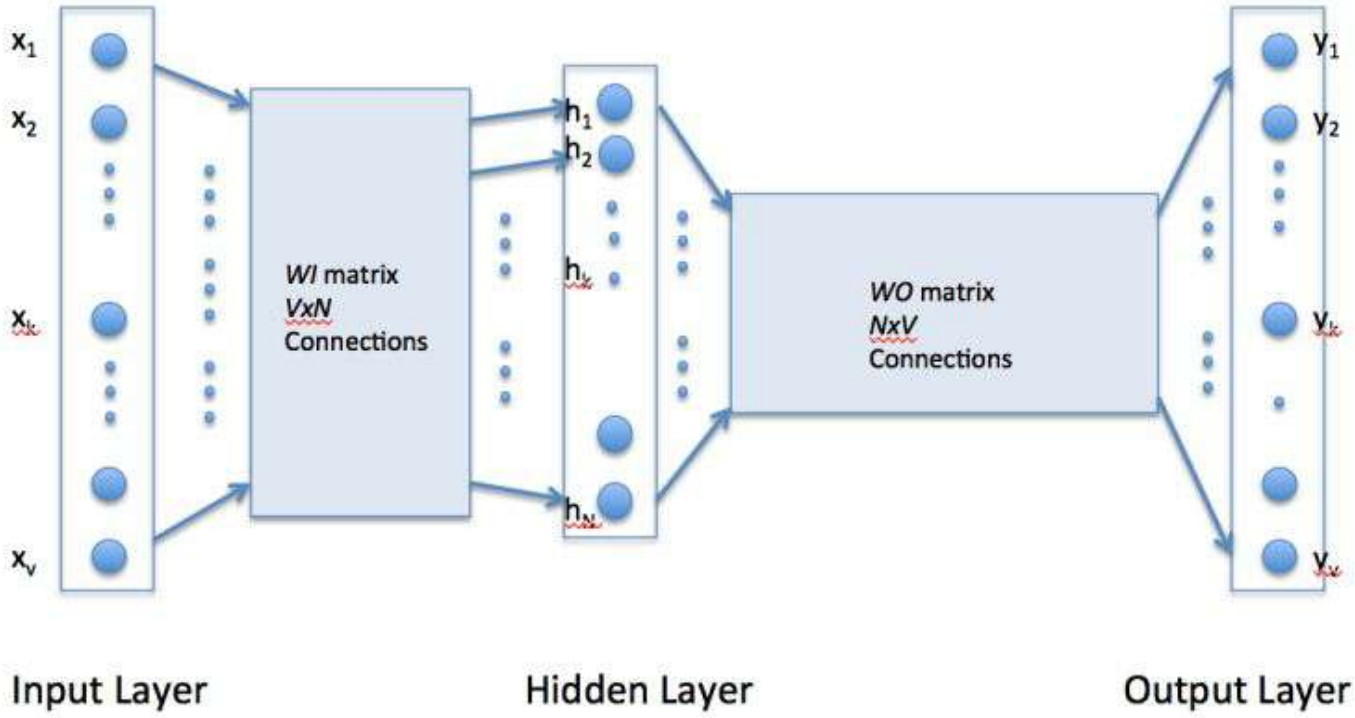
Skip-gram

- Two different optimization objectives:
  - Negative sampling (NS)
  - or hierarchical softmax

Word2vec

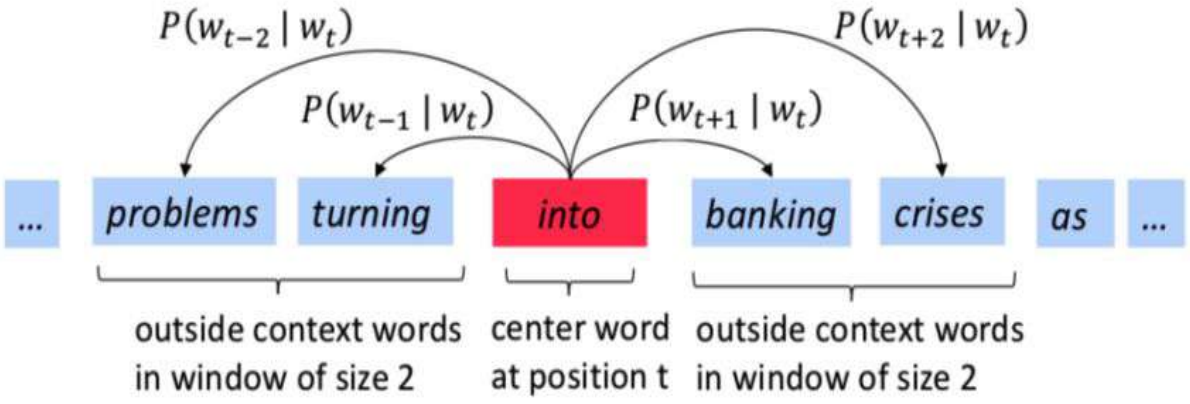
Two ways to think about Word2Vec:

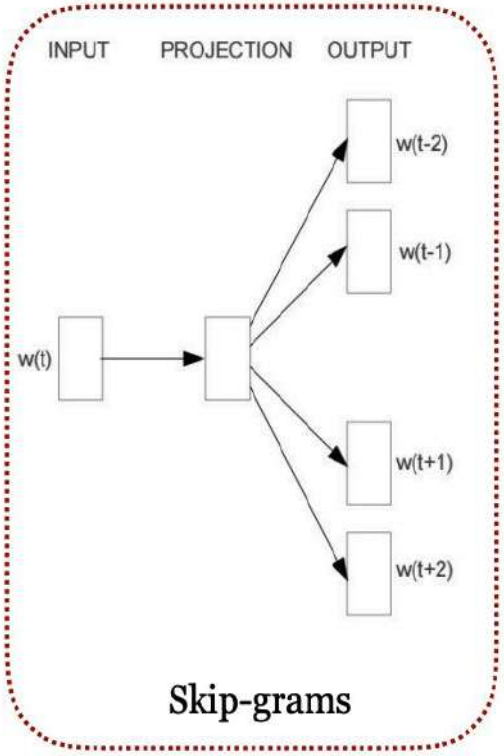
- a simplification of neural language models
- a binary logistic regression classifier



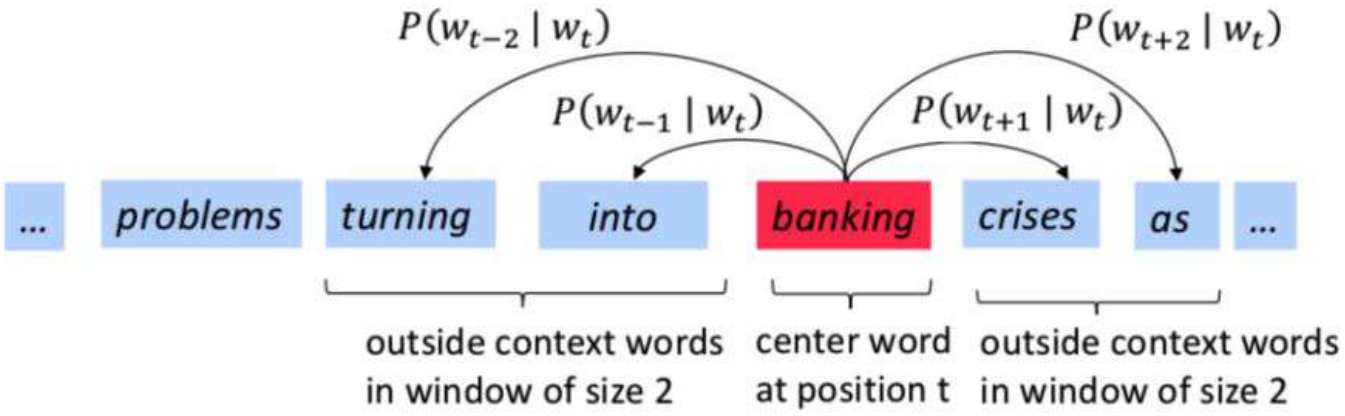
Word2vec with Skip-gram

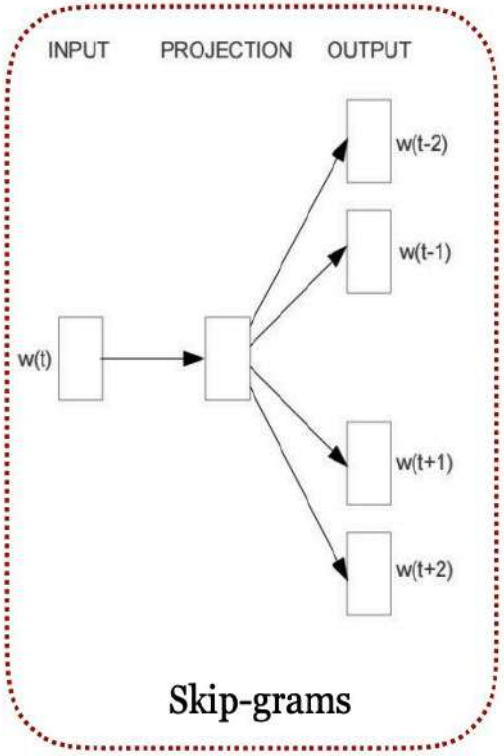
- The idea: we want to use words to **predict** their context words
- Context: a fixed window of size $2m$





Word2vec with Skip-gram





Word2vec with Skip-gram

- For each position $t = 1,2,\ldots T$, predict context words within context size m, given center word w_j:

all the parameters to be optimized

$$\mathcal{L}(\theta) = \prod_{t=1}^{T} \prod_{-m \le j \le m, j \neq 0} P(w_{t+j} \mid w_t; \theta)$$

- The objective function $J(\theta)$ is the (average) negative log likelihood:

$$J(\theta) = -\frac{1}{T} \log \mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \sum_{-m \le j \le m, j \neq 0} \log P(w_{t+j} \mid w_t; \theta)$$

Word2vec with Skip-gram

How to define $P(w_{t+j} \mid w_t; \theta)$?

- We have two sets of vectors for each word in the vocabulary

$\mathbf{u}_i \in \mathbb{R}^d$: embedding for target word i

$\mathbf{v}_{i'} \in \mathbb{R}^d$: embedding for context word i'

- Use inner product $\mathbf{u}_i \cdot \mathbf{v}_{i'}$ to measure how likely word i appears with context word i', the larger the better

"softmax"

$$P(w_{t+j} \mid w_t) = \frac{\exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_{w_{t+j}})}{\sum_{k \in V} \exp(\mathbf{u}_{w_t} \cdot \mathbf{v}_k)}$$

$\theta = \{\{\mathbf{u}_k\}, \{\mathbf{v}_k\}\}$ are all the parameters in this model!

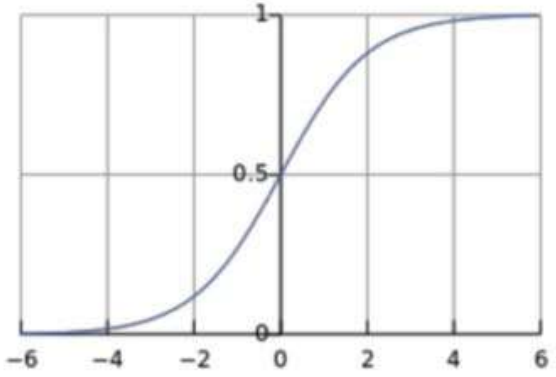
Word2vec with Skip-gram and Negative Sampling

Idea: recast problem as binary classification!

- Target word is positive example
- All words not in context are negative

$$P(D = 1 \mid t, c) = \sigma(\mathbf{u}_t \cdot \mathbf{v}_c)$$

$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$

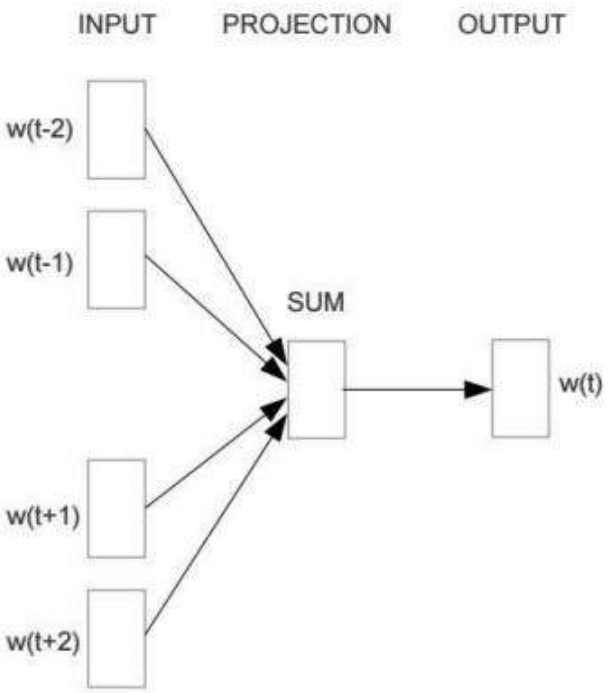


positive examples +		negative examples -			
t	c	t	c	t	c
apricot	tablespoon	apricot	aardvark	apricot	seven
apricot	of	apricot	my	apricot	forever
apricot	jam	apricot	where	apricot	dear
apricot	a	apricot	coaxial	apricot	if

To compute loss, pick K random words as negative examples:

$$J(\theta) = -P(D = 1 \mid t, c) - \frac{1}{K} \sum_{i=1}^{K} P(D = 0 \mid t_i, c)$$

Word2vec with Continuous Bag of Words (CBOW)



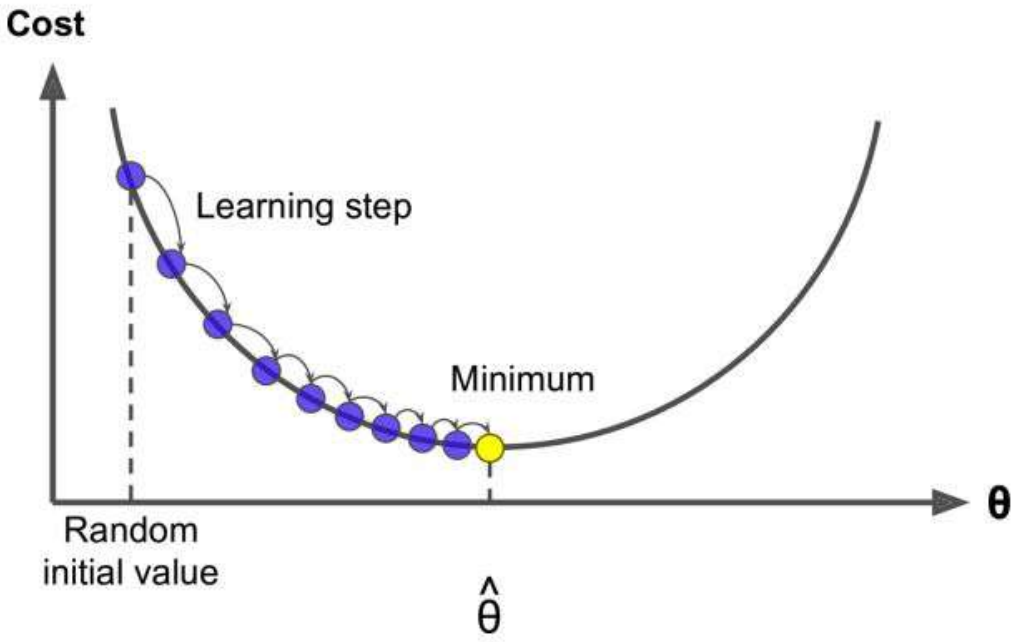
$$L(\theta) = \prod_{t=1}^{T} P\left(w_t \mid \{w_{t+j}\}, -m \leq j \leq m, j \neq 0\right)$$

$$\bar{\mathbf{v}}_t = \frac{1}{2m} \sum_{-m \leq j \leq m, j \neq 0} \mathbf{v}_{t+j}$$

$$P(w_t \mid \{w_{t+j}\}) = \frac{\exp(\mathbf{u}_{w_t} \cdot \bar{\mathbf{v}}_t)}{\sum_{k \in V} \exp(\mathbf{u}_k \cdot \bar{\mathbf{v}}_t)}$$

Refresher: Gradient descent

- We have a cost function $J(\theta)$ we want to minimize
- **Gradient Descent** is an algorithm to minimize $J(\theta)$
- **Idea:** for current value of θ, calculate gradient of $J(\theta)$, then take small step in direction of negative gradient. Repeat.



Note: Our objectives may not be convex like this ☹

But life turns out to be okay ☺

Refresher: Gradient descent

- Update equation (in matrix notation):

$$\theta^{new} = \theta^{old} - \alpha \nabla_\theta J(\theta)$$

α = *step size* or *learning rate*

- Update equation (for each single parameter θ_j):

$$\theta_j^{new} = \theta_j^{old} - \alpha \frac{\partial}{\partial \theta_j^{old}} J(\theta)$$

- Algorithm:

```
while True:
    theta_grad = evaluate_gradient(J,corpus,theta)
    theta = theta - alpha * theta_grad
```

In practice Stochastic Gradient Descent

- **Problem:** $J(\theta)$ is a function of **all** windows in the corpus (potentially billions!)
  - So $\nabla_\theta J(\theta)$ is very expensive to compute
- You would wait a very long time before making a single update!

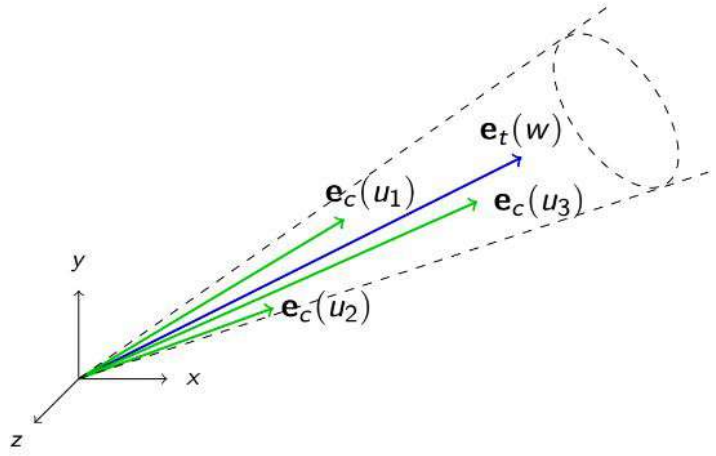
- **Very** bad idea for pretty much all neural nets!
- **Solution: Stochastic gradient descent (SGD)**
  - Repeatedly sample windows, and update after each one
- Algorithm:

Mini Batch Gradient Descent

```
while True:
    window = sample_window(corpus)
    theta_grad = evaluate_gradient(J,window,theta)
    theta = theta - alpha * theta_grad
```

Geometric interpretation (with a different notation)

Target vector $\mathbf{e}_t(w)$ is surrounded by its context vectors $\mathbf{e}_c(u_j)$, which form a "thin" hypercone (dashed lines)



Let w be a target word with context words u_j's. Let also w' be a target word similar to w.

Then we have that

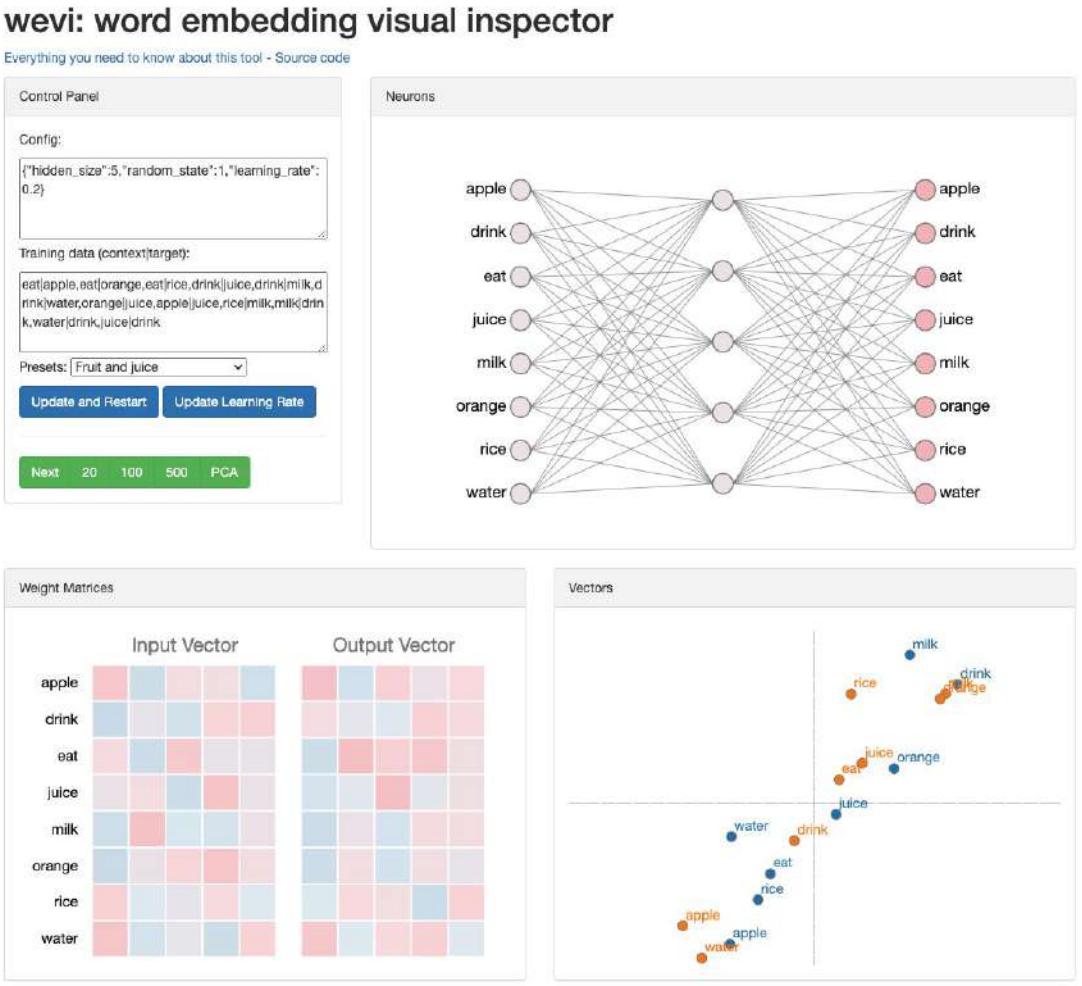
- $\mathbf{e}_c(u_j)$ form a thin cone surrounding vector $\mathbf{e}_t(w)$
- vector $\mathbf{e}_t(w')$ must be placed within that cone, because w' and w share their context words
- $\mathbf{e}_t(w)$ and $\mathbf{e}_t(w')$ are forced to be at a **small angle**

As a result, similar words cluster together.

With a similar reasoning, we can argue that different words repel each other.

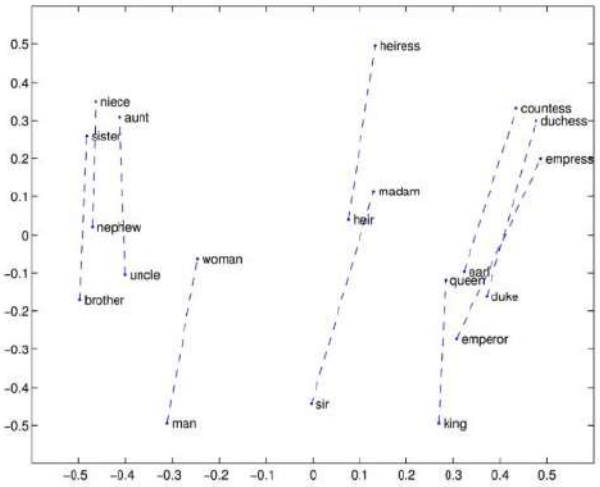
Word2vec demonstration

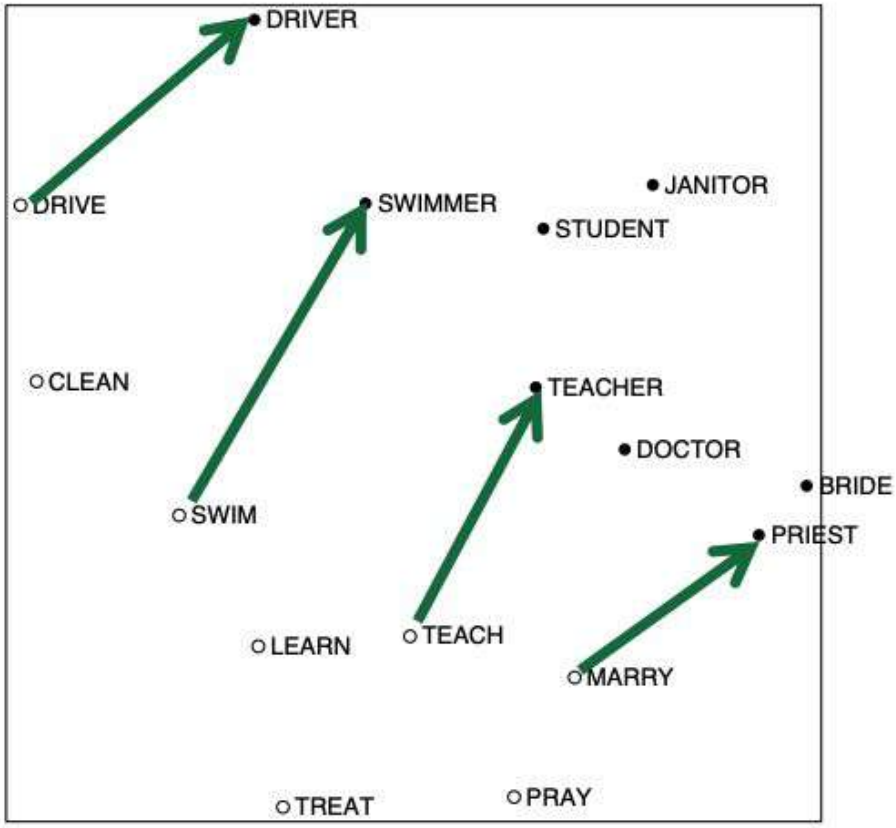
https://ronxin.github.io/wevi/



Properties of Word2vec (and other embeddings)

Interesting semantic patterns emerge in the scaled vectors

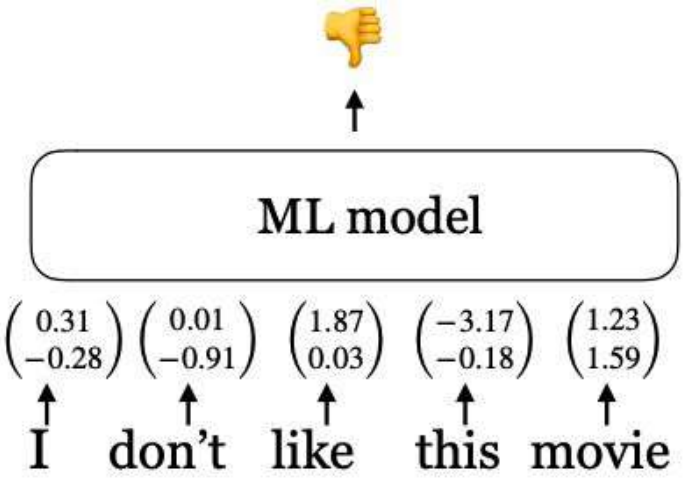




How do I know if my embedding is good?

Extrinsic evaluation

- Let's plug these word embeddings into a real NLP system and see whether this improves performance
- Could take a long time but still the most important evaluation metric



Intrinsic evaluation

- Evaluate on a specific/intermediate subtask
- Fast to compute
- Not clear if it really helps the downstream task

Intrinsic evaluation

Word similarity

Example dataset: wordsim-353
353 pairs of words with human judgement
http://www.cs.technion.ac.il/~gabr/resources/data/wordsim353/

Word 1	Word 2	Human (mean)
tiger	cat	7.35
tiger	tiger	10
book	paper	7.46
computer	internet	7.58
plane	car	5.77
professor	doctor	6.62
stock	phone	1.62
stock	CD	1.31
stock	jaguar	0.92

Cosine similarity:

$$\cos(\boldsymbol{u}_i, \boldsymbol{u}_j) = \frac{\boldsymbol{u}_i \cdot \boldsymbol{u}_j}{||\boldsymbol{u}_i||_2 \times ||\boldsymbol{u}_j||_2}.$$

Metric: Spearman rank correlation

FastText

Accomplishments:

- Words can be repesented as dense, low-dimensional vectors ✓
- Easy to capture similarity between words ✓
- Additive Compositionality of word vectors ✓

Open issues:

- Even if we train Word2Vec on a very large corpus, we will still encounter unknown words at test time
- What about rare words?
- Orthography can often help us:
- $\vec{w}^{(\text{remuneration})}$ should be similar to
  - $\vec{w}^{(\text{remunerate})}$ (same stem)
  - $\vec{w}^{(\text{iteration})}$, $\vec{w}^{(\text{consideration})}$ . . . (same suffix ≈ same POS)

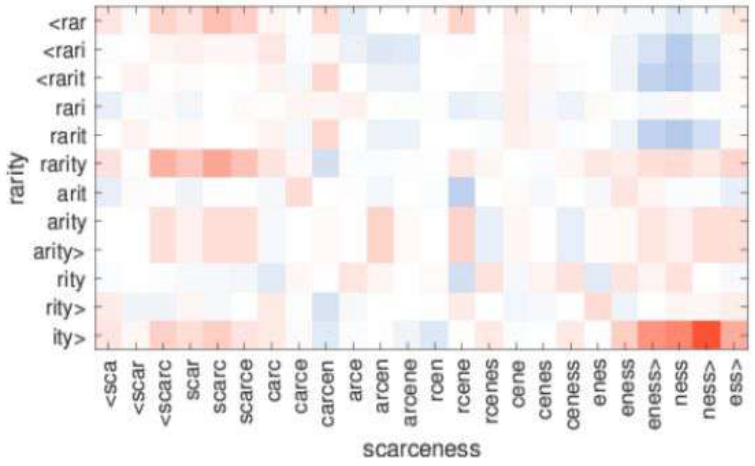
<exa, exam, xamp, ampl, mple, ple>,
<example>

- Note, that the 4-gram *exam* is different from the word <exam>.

FastText

fastText

- A vector is learned for each character n-gram
- Word are represented as the sum of their character n-grams vectors.
- Partially solves the out-of-vocabulary (OOV) words problem:
- unknown words still contain known n-grams.
- `https://fasttext.cc/`
- `http://vectors.nlpl.eu/repository/`



FastText

- ngrams typically contains character 3- to 6-grams
- Replace $\vec{w}$ in Skipgram objective with its new definition
- During backpropagation, loss gradient vector $\frac{\partial L}{\partial \vec{w}^{(i)}}$ is distributed to word vector $\vec{u}^{(i)}$ and associated n-gram vectors $\vec{u}^{(n)}$

Representation of a known word:
Average of the word's embedding and char-n-gram embeddings

$$\vec{w}^{(i)} = \frac{1}{|\text{ngrams}(i)| + 1}\left[\vec{u}^{(i)} + \sum_{n \in \text{ngrams}(i)} \vec{u}^{(n)}\right]$$

Representation of an unknown word:
Average of char-n-gram embeddings

$$\vec{w}^{(i)} = \frac{1}{|\text{ngrams}(i)|} \sum_{n \in \text{ngrams}(i)} \vec{u}^{(n)}$$

Assume, we want to represent the word *example*:

- Character n-grams (n = 3):

  <ex, exa, xam, amp, mpl, ple, le>, <example>

- In practice, we don't set $n = a$ but rather $a \leq n \leq b$
- Character n-grams ($2 \leq n \leq 4$):

  <e, ex, xa, am, mp, pl, le, e>,
  <ex, exa, xam, amp, mpl, ple, le>,
  <exa, exam, xamp, ampl, mple, ple>,
  <example>

- Note, that the 4-gram *exam* is different from the word <exam>

Other Work on Word Embeddings

- using subword information (e.g., characters) in word embeddings
- multiple embeddings for a single word type corresponding to different word senses
- tailoring embeddings using particular resources or for particular NLP tasks

Other Work on Word Embeddings

- using subword information (e.g., characters) in word embeddings

- multiple embeddings for a single word type corresponding to different word senses

- tailoring embeddings using particular resources or for particular NLP tasks

Subword Modeling for Word Embeddings

- Using word embeddings has limitations:
  - closed vocabulary (100k-300k words is typical)
  - large number of parameters! (100k * 300)
  - for morphologically-rich languages, using a separate vector for each word type is "obviously" wrong

- Solution: **character**-level modeling
  - open vocabulary, fewer parameters, often similar or better performance

Early Neural Methods

morphological analyzer + recursive neural network:

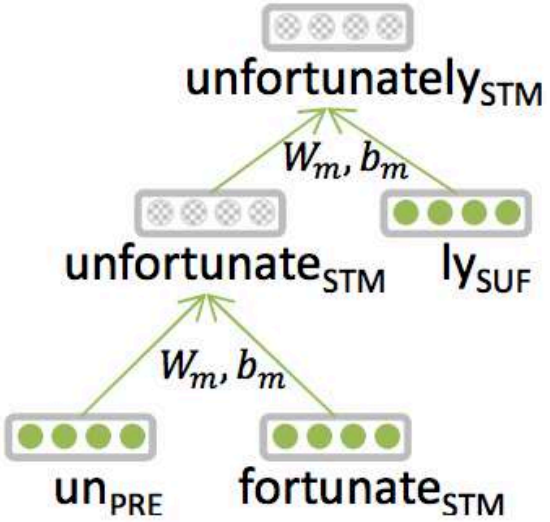


Figure 1: **Morphological Recursive Neural Network.** A vector representation for the word "unfortunately" is constructed from morphemic vectors: $un_{\texttt{pre}}$, $fortunate_{\texttt{stm}}$, $ly_{\texttt{suf}}$. Dotted nodes are computed on-the-fly and not in the lexicon.

Luong et al. (2013): *Better Word Representations with Recursive Neural Networks for Morphology*

unsupervised morphological analysis & vector addition:

$$\overrightarrow{\text{imperfection}} = \overrightarrow{im} + \overrightarrow{perfect} + \overrightarrow{ion}$$
$$\overrightarrow{\text{perfectly}} = \overrightarrow{perfect} + \overrightarrow{ly}.$$

We include the surface form of a word as a factor itself. This accounts for noncompositional constructions ($\overrightarrow{\text{greenhouse}} = \overrightarrow{greenhouse} + \overrightarrow{green} + \overrightarrow{house}$), and makes the approach more robust to noisy morphological segmentation. This strategy also overcomes the order-invariance of additive composition ($\overrightarrow{\text{hangover}} \neq \overrightarrow{\text{overhang}}$).

Botha & Blunsom (2014): *Compositional Morphology for Word Representations and Language Modelling*

visualization of learned morpheme vectors:

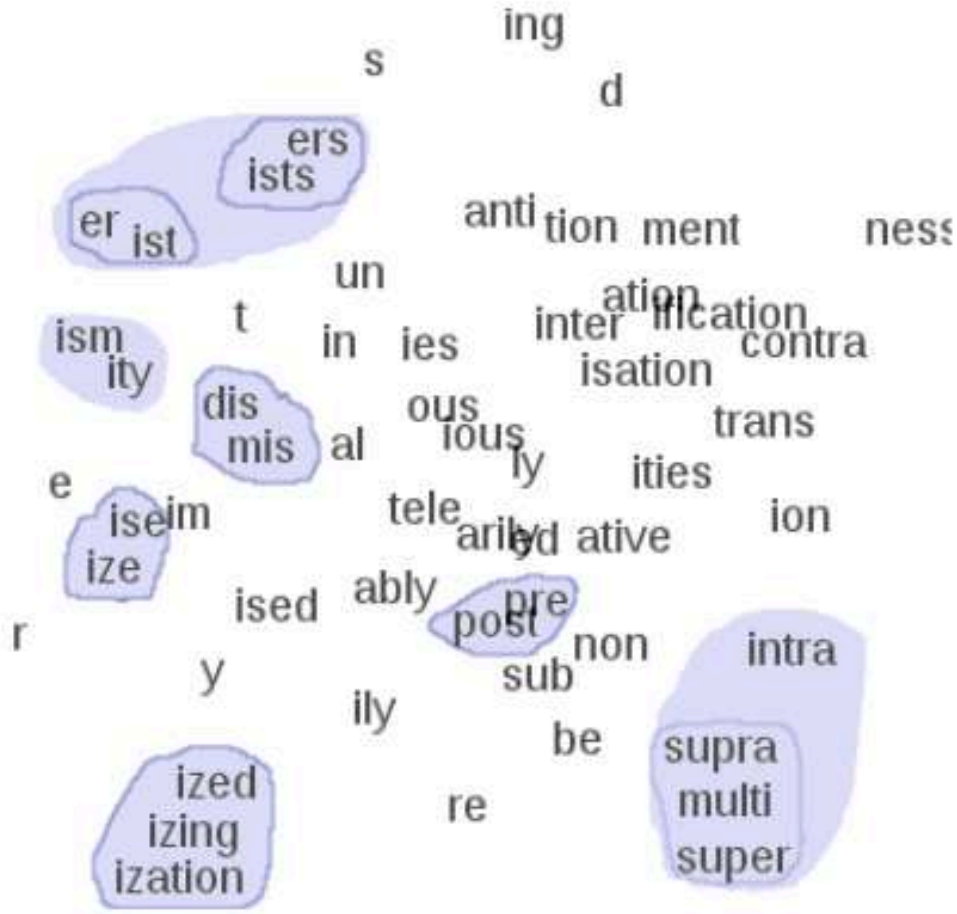
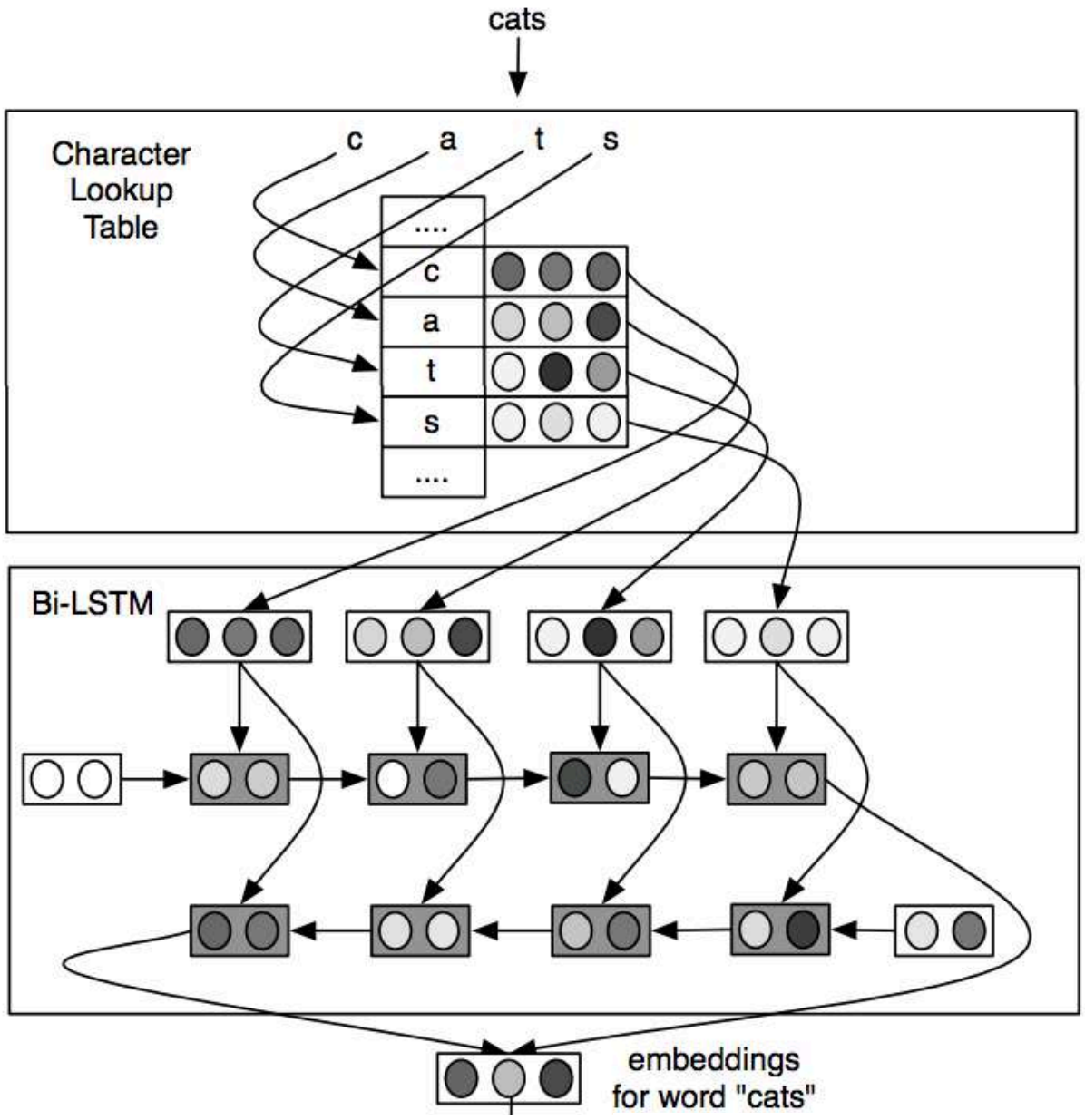


Figure 6. **English morpheme vectors learnt by CLBL++.**

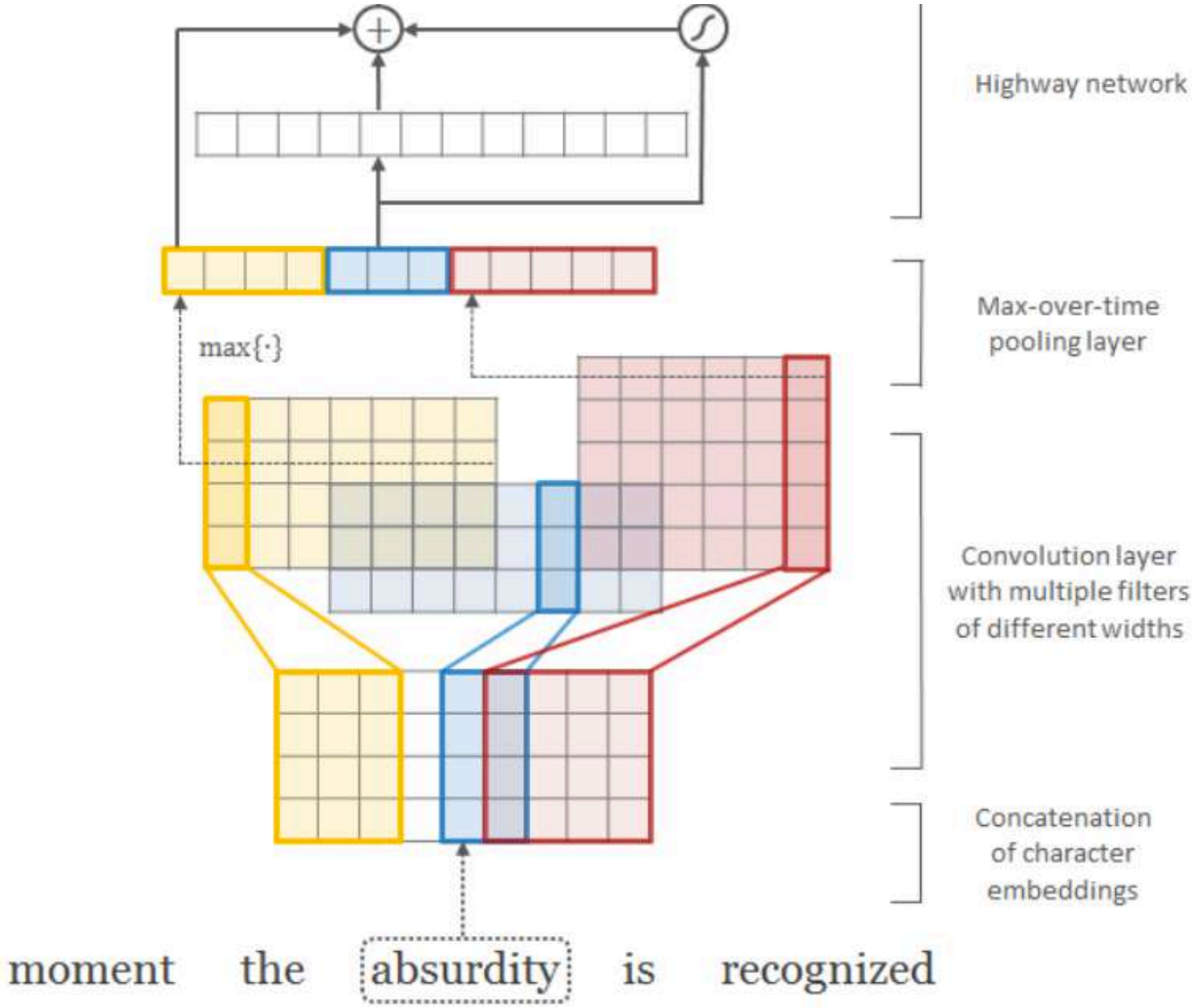
Botha & Blunsom (2014): *Compositional Morphology for Word Representations and Language Modelling*

Bidirectional LSTM over Characters



Ling et al. (2015): *Finding Function in Form: Compositional Character Models for Open Vocabulary Word Representation*

Convolutional Neural Network over Character Sequence



Kim et al. (2016): *Character-Aware Neural Language Models*

Convolution

$\boldsymbol{x} =$ *not that great*

$$\mathbf{x} = [\underbrace{0.4 \ \ldots \ 0.9}_{\text{vector for } \textit{not}} \ \underbrace{0.2 \ \ldots \ 0.7}_{\text{vector for } \textit{that}} \ \underbrace{0.3 \ \ldots \ 0.6}_{\text{vector for } \textit{great}}]^\top$$

$$c_1 = \mathbf{w}^\top \mathbf{x}_{1:d}$$

$$c_2 = \mathbf{w}^\top \mathbf{x}_{d+1:2d}$$

$$c_3 = \mathbf{w}^\top \mathbf{x}_{2d+1:3d}$$

Note: it's common to add a bias *b* and use a nonlinearity *g:*

$$c_1 = g\left(\mathbf{w}^\top \mathbf{x}_{1:d} + b\right)$$

Convolution

$\boldsymbol{x} =$ *not that great*

$$\mathbf{x} = [\underbrace{0.4 \ \ldots \ 0.9}_{\text{vector for } \textit{not}} \ \underbrace{0.2 \ \ldots \ 0.7}_{\text{vector for } \textit{that}} \ \underbrace{0.3 \ \ldots \ 0.6}_{\text{vector for } \textit{great}}]^\top$$

$$c_1 = \mathbf{w}^\top \mathbf{x}_{1:d}$$

$$c_2 = \mathbf{w}^\top \mathbf{x}_{d+1:2d}$$

$$c_3 = \mathbf{w}^\top \mathbf{x}_{2d+1:3d}$$

$\mathbf{c}$ = "feature map" for this filter,
has an entry for each position in input (in this case, 3 entries)

Pooling

$\boldsymbol{x} =$ *not that great*

how do we convert this into a fixed-length vector?
use **pooling**:

max-pooling: returns maximum value in $\mathbf{c}$

average pooling: returns average of values in $\mathbf{c}$

$$c_1 = \mathbf{w}^\top \mathbf{x}_{1:d}$$

$$c_2 = \mathbf{w}^\top \mathbf{x}_{d+1:2d}$$

$$c_3 = \mathbf{w}^\top \mathbf{x}_{2d+1:3d}$$

$\mathbf{c}$ = "feature map" for this filter,
has an entry for each position in input (in this case, 3 entries)

Pooling

$\boldsymbol{x} =$ *not that great*

how do we convert this into a fixed-length vector?
use **pooling**:

max-pooling: returns maximum value in $\mathbf{c}$

average pooling: returns average of values in $\mathbf{c}$

$$c_1 = \mathbf{w}^\top \mathbf{x}_{1:d}$$

$$c_2 = \mathbf{w}^\top \mathbf{x}_{d+1:2d}$$

then, this single filter $\mathbf{w}$ produces a single feature value (the output of some kind of pooling).

in practice, we use many filters of many different lengths (e.g., *n*-grams rather than words).

Convolutional Neural Networks

- "convolutional layer" = set of filters that are convolved with the input vector (whether ***x*** or hidden vector)
- could be followed by more convolutional layers, or by a type of pooling
- filters of varying n-gram lengths commonly used (1- to 5-grams)
- CNNs commonly used for character-level processing; filters look at character n-grams

- what about simpler methods?
- add or average vectors for character *n*-grams in the word:
  - word space (Schutze, 1993)
  - deep structured semantic models (Huang et al., 2013)
  - charagram (Wieting et al., 2016)
  - fastText (Bojanowski et al., 2017)

fastText

- like word2vec, but represents a word as the sum of its character *n*-gram embeddings and an embedding for the word itself

We also include the word w itself in the set of its n-grams, to learn a representation for each word (in addition to character n-grams). Taking the word *where* and $n = 3$ as an example, it will be represented by the character n-grams:

<wh, whe, her, ere, re>

and the special sequence

<where>.

Bojanowski et al. (2017): *Enriching Word Vectors with Subword Information*

For words in training set:

word	nearest neighbors
refunding	refunds, refunded, refund, repayment, reimbursement, rebate, repay reimbursements, reimburse, repaying, repayments, rebates, rebating
professors	professor, professorships, professorship, teachers, professorial, teacher prof., teaches, lecturers, teachings, instructors, headteachers
huge	enormous, tremendous, large, big, vast, overwhelming, immense, giant formidable, considerable, massive, huger, large-scale, great, daunting

For words not in training set:

word	nearest neighbors
vehicals	vehical, vehicles, vehicels, vehicular, cars, vehicle, automobiles, car
journeying	journey, journeys, voyage, trip, roadtrip, travel, tourney, voyages, road-trip
babyyyyyy	babyyyyyyyy, baby, babys, babe, baby.i, babydoll, babycake, darling

Wieting et al. (2016): *Charagram: Embedding Words and Sentences via Character n-grams*

fastText

- better data efficiency than word2vec:

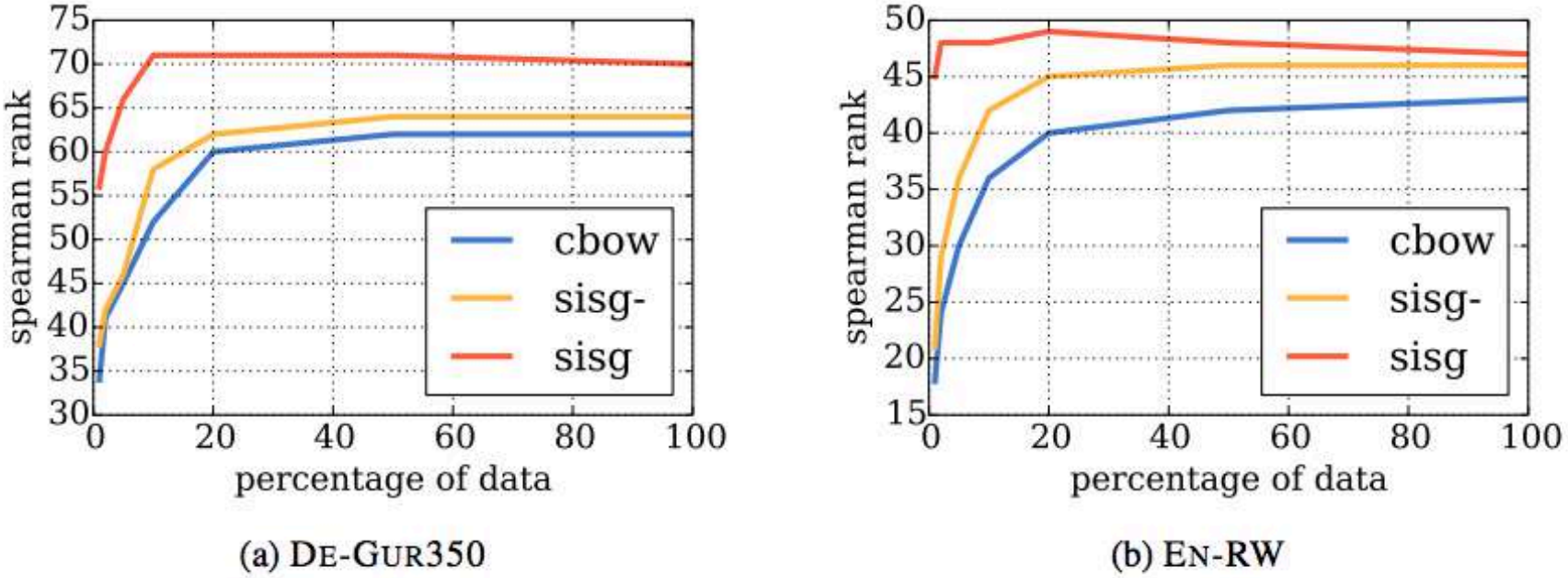


Figure 1: Influence of size of the training data on performance. We compute word vectors following the proposed model using datasets of increasing size. In this experiment, we train models on a fraction of the full Wikipedia dump.

Bojanowski et al. (2017): *Enriching Word Vectors with Subword Information*

- if you're just encoding text (rather than generating), you can use neural architectures like these to capture subword information

- for generation, it's trickier:
  - character RNNs are fine for generating words, but not sentences (very long sequences and long-distance dependencies)

- simple, data-driven segmentation methods have emerged as the standard way to handle this

Data-Driven Segmentation

- Most popular methods:
  - Byte pair encoding (BPE)
  - SentencePiece's unigram LM

SentencePiece



SentencePiece is an unsupervised text tokenizer and detokenizer mainly for Neural Network-based text generation systems where the vocabulary size is predetermined prior to the neural model training. SentencePiece implements **subword units** (e.g., **byte-pair-encoding (BPE)** [Sennrich et al.]) and **unigram language model** [Kudo.]) with the extension of direct training from raw sentences. SentencePiece allows us to make a purely end-to-end system that does not depend on language-specific pre/postprocessing.

This is not an official Google product.

Technical highlights

Data-Driven Segmentation

- Most popular methods:
  - Byte pair encoding (BPE)
  - SentencePiece's unigram LM

- these are easy and fast to use & work well

- they permit unbounded vocabularies with a relatively small number of parameters

Byte Pair Encoding (BPE)

(Gage, 1994)

- simple data compression technique
- iteratively replaces most frequent pair of bytes in a sequence with a single, unused byte
- Sennrich et al. (2016) adapted BPE for characters and character sequences

Byte Pair Encoding (BPE)

- **merge**: a rule that combines two consecutive units into a single unit
- initially, units are characters
- after merges, units become character sequences
- greedy algorithm:
  - merge two units with the largest unit bigram count, produce merged unit
  - replace all instances of that 2-unit sequence with the merged unit, recompute counts

Byte Pair Encoding (BPE)

- Sennrich et al. use BPE based on word counts from a corpus
  - sentences are not used; all that's needed are word types and their counts
  - special treatment for end-of-word symbol </w> (an unseen initial step merges final character in each word with </w>)
  - when segmenting new data, segments words individually (does not use context)

Corpus:

```
low
low
lower
lowest
high
high
higher
```

Merges:

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:

```
low</w>
low</w>
lower</w>
lowest</w>
high</w>
high</w>
higher</w>
```

Merges:

actually the corpus looks like this

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:

```
low</w>
low</w>
lower</w>
lowest</w>
high</w>
high</w>
higher</w>
```

Merges:

```
w </w>   (2)
r </w>   (2)
h </w>   (2)
t </w>   (1)
```

the first thing we do is merge word-ending characters with </w>

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:

```
low</w>
low</w>
lower</w>
lowest</w>
high</w>
high</w>
higher</w>
```

Segmentation:

Merges:

```
w </w>  (2)
r </w>  (2)
h </w>  (2)
t </w>  (1)
```

given this set of merges, let's segment the corpus!

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:	Segmentation:	Merges:
low</w>	l o w</w>	w </w> (2)
low</w>	l o w</w>	r </w> (2)
lower</w>	l o w e r</w>	h </w> (2)
lowest</w>	l o w e s t</w>	t </w> (1)
high</w>	h i g h</w>	
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

Sennrich et al. (2016): *Neural Machine Translation of Rare Words with Subword Units*

Corpus:	Segmentation:
low</w>	l o w</w>
low</w>	l o w</w>
lower</w>	l o w e r</w>
lowest</w>	l o w e s t</w>
high</w>	h i g h</w>
high</w>	h i g h</w>
higher</w>	h i g h e r</w>

Merges:

```
w </w> (2)
r </w> (2)
h </w> (2)
t </w> (1)
```

looking at the segmented corpus shows us what merge will occur next

Corpus:	Segmentation:	Merges:
low</w>	l o w</w>	w </w> (2)
low</w>	l o w</w>	r </w> (2)
lower</w>	l o w e r</w>	h </w> (2)
lowest</w>	l o w e s t</w>	t </w> (1)
high</w>	h i g h</w>	
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

what is the next merge?
(what unit bigram appears most often?)

Sennri… …re
Words …

Corpus:	Segmentation:	Merges:
low</w>	l o w</w>	w </w> (2)
low</w>	l o w</w>	r </w> (2)
lower</w>	l o w e r</w>	h </w> (2)
lowest</w>	l o w e s t</w>	t </w> (1)
high</w>	h i g h</w>	l o (4)
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

given this new set of merges, let's re-segment the corpus!

Sennrich et al. (2
Words with Subw

Corpus:	Segmentation:
low</w>	lo w</w>
low</w>	lo w</w>
lower</w>	lo w e r</w>
lowest</w>	lo w e s t</w>
high</w>	h i g h</w>
high</w>	h i g h</w>
higher</w>	h i g h e r</w>

Merges:

```
w </w>  (2)
r </w>  (2)
h </w>  (2)
t </w>  (1)
l o     (4)
```

Sennrich et al. (2016): *Neural M*
Words with Subword Units

note: we will always "back off" to the complete segmentation

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h i g h</w>	l o (4)
high</w>	h i g h</w>	
higher</w>	h i g h e r</w>	

what is the next merge?
(what unit bigram appears most often?)

Sennrich et al., 2016, *Neural Machine Translation of Rare Words with Subword Units*

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h i g h</w>	l o (4)
high</w>	h i g h</w>	i g (3)
higher</w>	h i g h e r</w>	

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h ig h</w>	l o (4)
high</w>	h ig h</w>	i g (3)
higher</w>	h ig h e r</w>	

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	h ig h</w>	l o (4)
high</w>	h ig h</w>	i g (3)
higher</w>	h ig h e r</w>	h ig (3)

Corpus:	Segmentation:	Merges:
low</w>	lo w</w>	w </w> (2)
low</w>	lo w</w>	r </w> (2)
lower</w>	lo w e r</w>	h </w> (2)
lowest</w>	lo w e s t</w>	t </w> (1)
high</w>	hig h</w>	l o (4)
high</w>	hig h</w>	i g (3)
higher</w>	hig h e r</w>	h ig (3)

Corpus:	Segmentation:
low</w>	low</w>
low</w>	low</w>
lower</w>	lowe r</w>
lowest</w>	lowe s t</w>
high</w>	high</w>
high</w>	high</w>
higher</w>	hig h e r</w>

Merges:

```
w </w>  (2)
r </w>  (2)
h </w>  (2)
t </w>  (1)
l o     (4)
i g     (3)
h ig    (3)
w e     (2)
lo we   (2)
lo w</w>(2)
hig h</w>(2)
```

we'd like to merge `e` and `r</w>`, but we merged `w` and `e` already, so that messed us up

New Corpus:

```
low (2x)
lower
lowest
high (2x)
higher
small
smaller
smallest
```

Merges for New Corpus:

```
l o
s m
sm a
sma l
i g
h ig
e r</w>
smal l
s t</w>
lo w</w>
lo w
hig h</w>
e st</w>
```

Merges:

```
l o
s m
sm a
sma l
i g
h ig
e r</w>
smal l
s t</w>
lo w</w>
lo w
hig h</w>
e st</w>
```

Application:

low → low

lower → low er

lowest → low est

high → high

higher → hig h er

highest → hig h est

small → smal l

smaller → small er

smallest → small est

- we can limit the vocabulary size of the segmented data by limiting the number of merges

- this can be very helpful for handling an open vocabulary of words while reducing computation (e.g., when using a softmax over the vocabulary)

Stanford Sentiment Treebank

BPE merging on train+dev sets

up to 20k merges (max number found: 15,417)

Example from test set:

`writer/director/producer` →

`writ@@ er@@ /@@ direct@@ or@@ /@@ producer`

(To recover original text, remove "`@@ `")

```
writ@@ er@@ /@@ direct@@ or@@ /@@ producer

Here 's a British flick gle@@ efully un@@
concerned with plau@@ sibility , yet just
as determined to entertain you .
```

probably good: “unconcerned” becomes “un concerned”
maybe bad: “gle efully”

```
It would n't be my prefer@@ red way of
sp@@ ending 100 minutes or $ 7@@ .@@ 00 .
```

probably good: “prefer” is related to “preferred”
maybe bad: “spending” is not related to “ending”

Other Work on Word Embeddings

- using subword information (e.g., characters) in word embeddings

- multiple embeddings for a single word type corresponding to different word senses

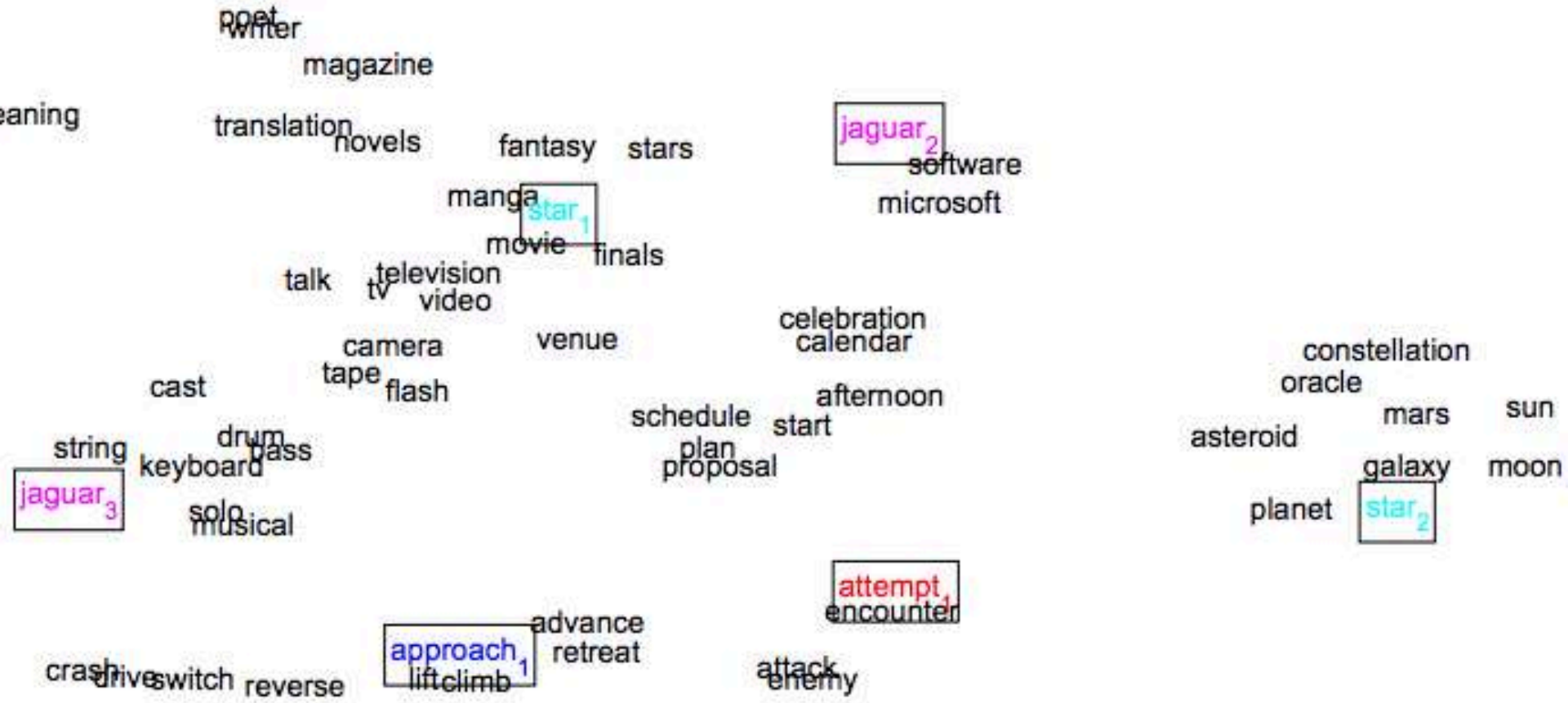
- tailoring embeddings using particular resources or for particular NLP tasks

Multisense Word Embeddings

- one embedding for a word type is insufficient
  - due to different senses of a word, different meanings (polysemy, homonymy)

- there has been a lot of work in learning sense-specific word embeddings:
  - use a word sense labeler or cluster word tokens into clusters that capture word sense
  - learn embeddings for each sense/cluster

Multisense Word Embeddings



Huang et al. (2012): *Improving Word Representations Via Global Context And Multiple Word Prototypes*

- nearest neighbors given context:

Context	Nearest Neighbors
Apple is a kind of fruit.	pear, cherry, mango, juice, peach, plum, fruit, cider, apples, tomato, orange, bean, pie
Apple releases its new ipads.	microsoft, intel, dell, ipad, macintosh, ipod, iphone, google, computer, imac, hardware
He borrowed the money from **banks**.	banking, credit, investment, finance, citibank, currency, assets, loads, imf, hsbc
along the shores of lakes, **banks** of rivers	land, coast, river, waters, stream, inland, area, coasts, shoreline, shores, peninsula
Basalt is the commonest volcanic **rock**.	boulder, stone, rocks, sand, mud, limestone, volcanic, sedimentary, pelt, lava, basalt
Rock is the music of teenage rebellion.	band, pop, bands, song, rap, album, jazz. blues, singer, hip-pop, songs, guitar, musician

Table 2: Nearest neighbors of words given context. The embeddings from context words are first inferred with the Greedy strategy; nearest neighbors are computed by cosine similarity between word embeddings. Similar phenomena have been observed in earlier work (Neelakantan et al., 2014)

Neelakantan et al. (2014): *Efficient nonparametric estimation of multiple embeddings per word in vector space*

Li & Jurafsky (2015): *Do Multi-Sense Embeddings Improve Natural Language Understanding?*

Multisense Word Embeddings

- limitations:
  - need a way to label senses or cluster word tokens in training data (and for downstream tasks)
  - fragments training data, so more may be needed for estimating word embeddings
  - unlikely to get good clusters for rare word types
  - unable to handle new senses that only appear in test data
  - unclear if sense-specific embeddings are useful for downstream tasks

Do Multisense Embeddings Help on NLP Tasks?

- yes, on some tasks

- but when using powerful neural architectures, multisense embeddings may not be needed

> We then test the performance of our model on part-of-speech tagging, named entity recognition, sentiment analysis, semantic relation identification and semantic relatedness, controlling for embedding dimensionality. We find that multi-sense embeddings do improve performance on some tasks (part-of-speech tagging, semantic relation identification, semantic relatedness) but not on others (named entity recognition, various forms of sentiment analysis).

- increasing dimensionality of (single-sense) embeddings achieves some benefit of multisense embeddings
  - high dimensionality also may make it easier for subsequent architectures to extract relevant sense based on context

Li & Jurafsky (2015): *Do Multi-Sense Embeddings Improve Natural Language Understanding?*